Access Control Conflict Resolution in Distributed File Systems using CRDTs

Elena Yanakieva
TU Kaiserslautern
Kaiserslautern, Germany
e_yanakiev15@cs.uni-kl.de

Michael Youssef
TU Kaiserslautern
Kaiserslautern, Germany
m_youssef19@cs.uni-kl.de

Ahmad Hussein Rezae
TU Kaiserslautern
Kaiserslautern, Germany
a_rezae19@cs.uni-kl.de

Annette Bieniusa
TU Kaiserslautern
Kaiserslautern, Germany
bieniusa@cs.uni-kl.de

ABSTRACT

Distributed file systems have become an essential service for sharing data among users. An important aspect of a file system is its ability to keep its contents secure from unauthorized access. To investigate the interplay of security and consistency in distributed file systems, we formalize the three properties related to data security, namely confidentiality, integrity and accessibility. Based on these properties, we provide an impossibility result that indicates that these properties cannot be achieved together in a highly-available partition-tolerant setting. We further discuss a CRDT-based model, implementing the traditional POSIX access control policy, that guarantees confidentiality and integrity while precluding accessibility only in rare situations. Our conclusion is that the POSIX policies are not suitable in a distributed system setting, but that a more fine-grained model is required to obtain the security semantics that reflect the users' intention.

CCS CONCEPTS

• **Computer systems organization → Distributed architectures**; • **Security and privacy → Access control**; **File system security**.

KEYWORDS

distributed file system, access control, conflict-free replicated data type, POSIX

ACM Reference Format:
Elena Yanakieva, Michael Youssef, Ahmad Hussein Rezae, and Annette Bieniusa. 2021. Access Control Conflict Resolution in Distributed File Systems using CRDTs. In *8th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC'21), April 26, 2021, Online, United Kingdom.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3447865.3457970

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
PaPoC'21, April 26, 2021, Online, United Kingdom
© 2021 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-8338-7/21/04.
https://doi.org/10.1145/3447865.3457970

1 INTRODUCTION

Distributed shared file systems have become an essential service for sharing data among users. To increase usability, on the one hand, such systems need to be designed to be highly available. Conflict-free replicated data types (CRDTs) have proven to guarantee high availability and have been successfully applied in the context of collaborative text editors. Meanwhile, they have also attracted attention in the context of distributed shared file systems. For example, the InterPlanetary File System (IPFS) [1] provides a distributed file system abstraction based on a P2P network; files are here structured into blocks and for each block, the different versions are accessible as a directed acyclic graph (DAG). Updates add new versions to the DAG, and branches in the graph can be conflated with new versions or via CRDTs.

On the other hand, a shared file system has to offer security mechanisms to prevent unauthorized access. A correct access control is required to prevent data leakage, but also to guarantee accessibility for authorized users. However, the focus in most research on CRDT-based file systems has been on the challenge to guarantee the integrity of the directory structure by preventing cycles in the tree structure due to concurrent move operations [1, 4–6]. Access control is usually handled in less detail and separately from the content of the files. For example, IPFS makes use of cryptographic encryption of new versions and delegates the accessibility problem to a key management subsystem. However, to our knowledge, there is little research on what security benefits and impacts is provided by handling access control metadata using merging strategies.

In this abstract, we argue that a CRDT design can ensure system security to a certain extent. Furthermore, we highlight the challenges and impossibilities that high-availability, coupled with security, poses.

2 AN ACCESS CONTROL MODEL FOR HIGHLY-AVAILABLE FILE SYSTEMS

Before we introduce our access control model for highly-available file systems, we introduce some central notions that it relies on. File systems store, organize and present data in a hierarchical structure. The data is represented by the files and organized in directories. It is important to note here that we assume that files and directories have a unique identifier.

[1] https://docs.ipfs.io/concepts/file-systems/#unix-file-system-unixfs

To provide good user experience, file systems should be highly available. A well-established approach for achieving high availability in distributed systems are Conflict-free replicated data types (CRDTs). They guarantee strong eventual consistency, meaning that replicas having received the same updates have the same state.

It is furthermore important for a file system to be able to keep its data secure. We say a system is secure when the following three properties are guaranteed: confidentiality, integrity and accessibility [2]. Confidentiality and integrity ensure that unauthorized read and write access is forbidden. Accessibility guarantees that authorized access is always allowed. In order to guarantee these properties, systems implement some access control policy. Since our work has its focus on distributed file system, we have chosen to look into the POSIX system access control design. This design offers three types of access permissions - read, write and execute, and three types of authorization - owner, group and others. The owner set contains a single user; by default, it is the user that created the entity. The owner is allowed to make changes to the access permissions for an entity. The group set contains a single group, which can comprise multiple users. The default group is the one, which consists only of the user that created the entity. Every other user in the file system that is neither the owner, nor part of the group, belongs to the other set. For each authorization set, a type of access permissions is defined, which can be *read*, *write*, *execute*, a mixture of those three or none. In our simplified model, we focus here only on read and write permissions.

To study the semantics of merging access control metadata, we designed a simplified file system model based on CRDTs that uses POSIX access control [7]. It consists of one directory and can have multiple files, users and groups. We formalized our model in Repliss, a testing and verification tool for highly available applications, which offers implemented CRDT types [8]. To this end, we extended the CRDT library with a custom datatype and the following policies [9]: Administrators never intend to harm the system; therefore, conflicts between administrators and users result in favor of the admin decision. When resolving conflicts between users, the resulting policy must comprise all restrictions. We have further formalized the following functions: change owner and group; change permissions for owner, group and others; create, read and write files; create users and groups and assign a user to a group. Finally, we defined two types of invariants - confidentiality/integrity and accessibility, which we want to verify and test against. Though the model applies some simplifications, we argue that it generates all relevant counterexamples:

- Reading and writing of files behaves similarly with respect to access control. Our model tracks the access, but not whether a file has been modified. Therefore, integrity and confidentiality coincide.
- Each authorization group behaves the same way. Owners differ with their ability to change permissions. Therefore, by verifying confidentiality and accessibility against the owner, the policies for group and others are also verified.

[2]In literature, accessibility is usually called availability. We have chosen the name accessibility to avoid confusion with the term "system availability".

The confidentiality invariant ensures that the reading rights have been taken away before an unsuccessful read, whereas the availability invariant guarantees that reading rights have been granted before a successful read.

When testing our model with Repliss, we derived two main conclusions. First, the access control CRDT must be atomic, that is, all information relevant to the access control needs to be accessible at conflict merge time. Otherwise, the access control metadata might converge in an undesirable way. For example, if the file owner and the file owner's permissions are changed concurrently, when updating the data individually the file obtains the new owner with the new permissions. This might conflict with the intended semantics of the operations, namely giving the former owner more rights and providing a new owner with unchanged rights. Second, there cannot exist a system that is highly available, partition tolerant and guarantees at the same time confidentiality, integrity and accessibility. This follows from the fact that the system is not guaranteed to be strongly consistent, due to the CAP theorem [2, 3]. By choosing different merging semantics, more or less restrictive, the system designer can influenced whether the system guarantees either accessibility or confidentiality and integrity. Though confidentiality and integrity can be achieved by introducing further mechanisms, such as cryptographic encryption, we argue that this does not provide better security while it prevents the system from being accessible.

3 CONCLUSION AND FUTURE WORK

Distributed file systems require security policies that reflect the users' intention without compromising availability under system partition. To investigate properties of typical security mechanisms, we modeled and checked confidentiality, integrity, and accessibility against a simplified CRDT-based file system model formalized in Repliss.

The testing results show situations, for which it is impossible to guarantee both confidentiality and accessibility. This indicates that system designers are faced with the decision whether accessibility or confidentiality/integrity should be sacrificed. Furthermore, having a system design with multiple non-atomic CRDTs leads to undesirable results due to the inability to define merging semantics considering a set of semantically related CRDTs. To overcome these limitations, we plan to define a custom CRDT that combines all of the semantically related access control data. This would mean that the defined merging semantics will be able to consider all the necessary information.

Our simplified model consists of only one directory. Even though we argue that our scenario depicts the most important challenges for such a system, having directories may introduce more conflict edge cases. For example, assume that owner A of directory D changes D's permissions to exclude user B from accessing D's contents. B concurrently copies a file from D to another directory E, to which B has access. This conflict can be considered an example of confidentiality violation by transitivity because B can now read copied files' content. Our model can be extended to detect such conflict cases.

Finally, we plan to enhance an existing distributed file system with the implementation of our model. It will give opportunities to test it in a realistic environment.

REFERENCES

[1] Mehdi Ahmed-Nacer, Stéphane Martin, and Pascal Urso. 2012. File system on CRDT. *CoRR* abs/1207.5990 (2012). arXiv:1207.5990 http://arxiv.org/abs/1207.5990

[2] Eric A. Brewer. 2000. Towards Robust Distributed Systems (Abstract). In *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing* (Portland, Oregon, USA) *(PODC '00)*. Association for Computing Machinery, New York, NY, USA, 7. https://doi.org/10.1145/343477.343502

[3] Seth Gilbert and Nancy Lynch. 2002. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *SIGACT News* 33, 2 (June 2002), 51–59. https://doi.org/10.1145/564585.564601

[4] Martin Kleppmann, Dominic P Mulligan, Victor BF Gomes, and Alastair R Beresford. 2020. A highly-available move operation for replicated trees and distributed filesystems.

[5] Mahsa Najafzadeh, Marc Shapiro, and Patrick Eugster. 2018. Co-Design and Verification of an Available File System. In *Verification, Model Checking, and Abstract Interpretation - 19th International Conference, VMCAI 2018, Los Angeles, CA, USA, January 7-9, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10747)*, Isil Dillig and Jens Palsberg (Eds.). Springer, 358–381. https://doi.org/10.1007/978-3-319-73721-8_17

[6] Vinh Tao, Marc Shapiro, and Vianney Rancurel. 2015. Merging semantics for conflict updates in geo-distributed file systems. In *Proceedings of the 8th ACM International Systems and Storage Conference, SYSTOR 2015, Haifa, Israel, May 26-28, 2015*, Dalit Naor, Gernot Heiser, and Idit Keidar (Eds.). ACM, 10:1–10:12. https://doi.org/10.1145/2757667.2757683

[7] Michael Youssef, Annette Bieniusa, Ahmad Hussein Rezae, and Elena Yanakieva. 2021. crdt-filesystem. https://github.com/AntidoteDB/crdt-filesystem.

[8] Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. 2019. Combining State- and Event-Based Semantics to Verify Highly Available Programs. In *Formal Aspects of Component Software - 16th International Conference, FACS 2019, Amsterdam, The Netherlands, October 23-25, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 12018)*, Farhad Arbab and Sung-Shik Jongmans (Eds.). Springer, 213–232. https://doi.org/10.1007/978-3-030-40914-2_11

[9] Peter Zeller, Michael Youssef, Annette Bieniusa, Ahmad Hussein Rezae, and Elena Yanakieva. 2021. repliss-filesystem. https://github.com/AntidoteDB/repliss-filesystem.